

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 160. Recursive Functions

Lecture Summary

This lecture introduces **recursive functions** in Python, emphasizing their importance in problem solving and competitive coding. A recursive function is defined as one that calls itself during execution. The instructor explains the key components of recursion including the **recursive call** and the essential **base condition** that stops repeated calls to prevent infinite recursion. The lecture includes a detailed walkthrough of a simple recursive function that prints numbers from a given positive integer down to 1, illustrating the call stack and return phase in recursion.

Further, the lecture explores the classical example of calculating a factorial using recursion. The factorial concept is reviewed mathematically, then translated into a recursive Python function. The lecturer demonstrates the approach of defining recursive problems mathematically first, then converting them into code. The factorial function showcases how recursion works both in the calling phase (going deeper) and returning phase (combining results to a final output).

The lecture ends with encouragement to practice understanding and writing recursive functions, noting that many recursive functions encountered might be pre-existing and understanding them is a key challenge.

Key Concepts and Explanations

1. What is a Recursive Function?

- A **recursive function** is a function that calls itself within its own definition.
- Each call to the function processes a smaller or simpler subset of the original problem.

2. Structure of Recursive Functions

- **Recursive Call**: The function calls itself with a modified argument (usually closer to a stopping point).
- **Base Condition (Base Case)**: A vital stopping condition to prevent infinite recursion.
 - When the base case is met, the function stops making recursive calls.

3. Example of a Recursive Function: Printing Numbers Downwards

- Function `fun(n)`:

- If `n > 0`, print `n` and call `fun(n-1)`.
- If `n <= 0`, stop recursion (base case).
- Key Point: Two phases in recursion — calling phase (going deeper) and returning phase (coming back).

4. Recursion vs Loop

- Recursive functions are an alternative to loops.
- Some problems are easier to express with recursion, especially from a mathematical standpoint.
- However, loops are generally faster and more efficient in Python.

5. Factorial Concept (Mathematical Review)

- Factorial of `n` (written as `n!`) = $(1 \times 2 \times 3 \times \dots \times (n-1) \times n)$.
- E.g., $(5! = 1 \times 2 \times 3 \times 4 \times 5 = 120)$.

6. Recursive Definition of Factorial

- Recursive formula:

$$\begin{aligned} & \text{\\[} \\ & n! = n \times (n-1)! \\ & \text{\\]} \end{aligned}$$

- Base condition:

$$\begin{aligned} & \text{\\[} \\ & 0! = 1 \\ & \text{\\]} \end{aligned}$$

7. Recursive Factorial Function in Python

- If `n <= 0`, return 1.
- Else return `n * fact(n - 1)`.
- Recursive call reduces the problem until base case is reached.
- Upon returning, factorials multiply back up the call stack.

8. Understanding Recursion Internally

- The function calls itself repeatedly with decreasing parameters until it hits the base case.

- Then it returns back through the call stack, performing operations (like multiplication) on return.

9. Practical Advice

- Write recursive functions by first understanding the mathematical definition.
- Try to convert recursive functions to loops for better performance where needed.
- Focus on learning to read and understand recursive functions since many are pre-written.

Example Code and Use Cases

Example 1: Simple Recursive Function to Print Numbers from n to 1

```
def fun(n):  
    if n > 0:  
        print(n)  
        fun(n - 1)  
  
fun(3)
```

This function prints: 3, 2, 1. It calls itself decreasing the argument until the base case `n <= 0` stops the recursion.

Example 2: Recursive Function to Calculate Factorial

```
def fact(n):  
    if n <= 0: # Base condition  
        return 1  
    else:  
        return n * fact(n - 1)  
  
print(fact(5)) # Output: 120
```

This function calculates the factorial of an integer n using recursion. It returns 1 for zero or negative inputs, and otherwise multiplies n by factorial of n-1.

Key Takeaways

- A **recursive function calls itself** to simplify a problem into smaller sub-problems.
- Every recursive function must have a **base condition** to stop infinite recursion.

- Recursion involves two phases: **call phase** (going deeper) and **return phase** (coming back with results).

- **Recursive functions are an alternative to loops**, useful especially for problems naturally defined by recurrence relations.

- The **factorial** is a classic example of recursion:

$n! = n \times (n-1)!$ with $0! = 1$.

- Understanding recursion mathematically aids in writing recursive programs.

- Recursive functions are often slower than loops; **convert recursive logic to iterative where efficiency is critical**.

- Practice reading and tracing recursion to fully grasp its flow and stack behavior.

Section 1: Lecture Summary

This section was missing from model output.

Section 2: Key Concepts and Explanations

This section was missing from model output.

Section 3: Example Code and Use Cases

This section was missing from model output.

Section 4: Key Takeaways

This section was missing from model output.